

CS 263 Project Check-in Report

Yichuan Wang

Qiuyang Mang

April 21, 2026

Project: Automated Test Oracles Discovery for DBMS via Lean

Group Members: Yichuan Wang, Qiuyang Mang

Project Goal

The goal of our project is to build an automated pipeline that discovers diverse, high-quality test oracles—pairs of equivalent SQL queries—for finding logic bugs in mature Database Management Systems. We combine LLM-based generation for scalability and diversity with Lean 4 formal verification to guarantee semantic equivalence under bag semantics, eliminating the hallucination problem that plagues purely LLM-driven oracle generation. The goal has not substantially changed since we started, though we have sharpened the scope of what Lean is expected to verify (see below).

Progress So Far

Since the start of the project, we have set up the end-to-end pipeline skeleton: LLM-driven SQL pair generation with in-context learning, a SQL pool with diversity-based sampling, and an initial Lean verification layer that encodes queries as bag relations ($\text{Bag } \alpha := \alpha \rightarrow \text{Nat}$) and discharges equivalence goals with `funext` plus `simp`-based tactics. We have working proofs for the “easy” fragment—filters, conjunction/disjunction rewriting, and equi-joins under bag semantics—and a preliminary experiment loop that produced encouraging numbers (roughly a 30% successful-generation rate at small scale). The LLM generation and pool management went faster than expected; getting Lean to reliably close proofs that the LLM proposes has been harder, mainly because the LLM often emits tactic scripts that look plausible but miss a decidability instance or invoke a lemma that doesn’t exist, so we spend real effort on proof repair and prompt iteration.

Related Work and Scope (addressing Max’s question)

Max asked what techniques people use to formally verify SQL and noted that bag semantics alone are not enough for features like `ORDER BY` or `top-k`. This is a fair point. Our plan is to focus this project on the **bag-semantics fragment**: projection, selection with boolean combinations of predicates, equi-joins, set/bag union, and basic filter rewrites. Historically, most SQL equivalence work (e.g., Cosette, HoTTSQL, Mediator) operated on exactly this fragment because it is where equational reasoning is cleanest. For richer features such as aggregation, `GROUP BY`, and `ORDER BY`, a recent SIGMOD 2024 paper, “*SQLSolver: Proving SQL Equivalence Using Linear Integer Arithmetic*”, provides semantics and decision procedures that extend beyond bags, and we view this as the natural follow-on direction rather than something we will tackle in-scope.

What Remains and Rescoping

Remaining work:

1. finish the ZSS tree-edit-distance diversity metric over query plans to drive far-island sampling,
2. harden the Lean verifier so that proof failures are classified (non-equivalence vs. tactic gap) and fed back into the LLM for repair,
3. run the full 1000-iteration experiment and measure diversity and successful-oracle yield, and
4. actually feed verified oracle pairs into a real DBMS (likely SQLite or DuckDB) to see whether we surface any logic bugs.

We have not needed to rescope meaningfully—the bag-semantics restriction was always the plan, and Max’s comment has simply made us more explicit about it in the writeup. A successful project, concretely, looks like: a reproducible pipeline, a few hundred Lean-verified equivalent SQL pairs, diversity measurements showing the evolutionary loop helps, and ideally at least one confirmed discrepancy on a real DBMS.